"Injury Risk Prediction System for Athletes Using Performance Data"

CO2 ASSESSMENT TOOL 2

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

to the award of the degree of

BACHELOR OF TECHNOLOGY IN

INFORMATION TECHNOLOGY

Submitted by

192521491 - DILLI BABU N

Under the

Supervision of Dr. K

Anita Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

August 2025

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

DECLARATION

I, **Dilli Babu N 192521491,** of the Department of Information Technology, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **"Injury Risk Prediction System for Athletes Using Performance Data"** is the result of my own Bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Chennai

Date:

Signature of the Student

INDEX TABLE (TABLE OF CONTENTS)

1. Analyze System Requirements

The target system is an Injury Risk Prediction System for Athletes Using Performance Data. It serves as a unified sports-science platform for coaches, physiotherapists, sports scientists, and athletes to continuously monitor training load, biomechanical, and physiological data, and to generate early, data-driven warnings of impending injury risk before it results in time-loss injuries.

1.1. System Objectives

- Real-time Performance Monitoring: Ingest continuous streams of wearable and sensor data (GPS load, heart-rate variability, accelerometer/IMU, force-plate readings) with minimal latency from training sessions and matches.
- Predictive Risk Scoring: Continuously compute a per-athlete injury risk score using machine learning models trained on training load ratios (ACWR), fatigue markers, recovery metrics, and prior injury history.
- Early Warning Alerts: Automatically notify coaches and medical staff in real time when an athlete's risk score crosses a configurable threshold, enabling proactive load management.
- Historical Trend Analysis: Provide longitudinal dashboards comparing training load, recovery, and risk trends across seasons to support return-to-play and periodisation decisions.
- Secure Role-based Access Control (RBAC): Guarantee that coaches, physiotherapists, sports scientists, athletes, and administrators have strictly enforced, audit-logged read/write/comment capabilities over sensitive health data.
- Multi-source Data Integration: Enable ingestion from heterogeneous wearable ecosystems (Catapult, Polar, Whoop, Firstbeat), manual RPE entry, and clinical/medical record systems.

1.2. Functional Requirements (Influencing Architecture)

Functional requirements dictate the structural abstractions of our system. Below are key functional areas that directly dictate the architectural layout:

Functional Module	Requirement Details	Architectural Influence
Data Ingestion Engine	Continuously ingest performance and biometric data from multiple wearable APIs and device formats at high throughput during live sessions.	Requires a streaming ingestion pipeline (Kafka), per-device adapter microservices, and schema normalization workers.
ML Risk Prediction Engine	Compute a rolling injury-risk score per athlete using trained models (e.g., gradient boosting / LSTM) over sliding-window performance features.	Requires a dedicated ML inference microservice, an online feature store, and a versioned model-serving/registry infrastructure.
Alerting & Notification	Trigger real-time alerts to coach/physio dashboards and mobile push notifications when risk scores exceed thresholds.	Requires event-driven pub/sub messaging, a notification microservice, and WebSocket-based push delivery.

Functional Module	Requirement Details	Architectural Influence
Athlete Profile & History	Store athlete demographics, injury history, training logs, and support full CRUD with audit trails.	Requires a relational database with ACID transactions, REST APIs, and RBAC-enforced access at the API Gateway.

1.3. Non-Functional Requirements (NFRs)

NFR Category	Target Metric	Architectural Design Decision
Latency	Risk score recomputed within < 2 seconds of new sensor data arriving.	Stream processing via Kafka consumers, in-memory feature computation, and a Redis-backed online feature store.
Availability	99.9% availability for monitoring and alerting services during live training/competition.	Multi-region Kubernetes deployment, automated service discovery, and database failover with replica sets.
Scalability	Support thousands of athletes across hundreds of teams generating high-frequency sensor data concurrently.	Horizontally scalable ingestion/consumer nodes, partitioned Kafka topics, and a sharded time-series database.
Data Durability	Zero loss of sensor readings under sudden network disconnection at the training ground.	Local buffering at the device gateway with queued retries, Kafka topic replication, and acknowledgement-based delivery.
Security & Privacy	End-to-end transport encryption (TLS 1.3), OAuth 2.0 auth, and field-level encryption for athlete health data.	Kong API Gateway terminates TLS, verifies JWT signatures, and routes requests to microservices over a private VPC.

1.4. Key Quality Attributes

To provide a solid software architecture, we identify and optimize six core quality attributes:

- Scalability: The platform must scale to thousands of concurrently monitored athletes across many teams and venues. The architecture isolates high-frequency sensor ingestion from persistent databases using stateless gateways and cache-first, stream-processing workflows.
- Maintainability: Each sub-domain (Auth, Athlete Profile, Data Ingestion, Risk Prediction, Alerting, Analytics) is developed as a separate microservice, guaranteeing independent codebases so teams can retrain models or refactor schemas without risking global regression.
- Reliability: Data integrity of injury and health data is paramount. Sensor streams are replicated across Kafka partitions and acknowledged end-to-end so no reading is silently dropped, and model predictions are versioned for reproducibility.
- Availability: Redundant containerised nodes, automated service discovery, and database failover strategies guarantee continuous monitoring even if a database primary node fails mid-session.

- Performance: Coaches expect near-instant risk visibility. Dashboards use cached feature summaries while model inference and heavy analytics run as background/stream workers to protect UI responsiveness.
- Security: Deep transport-layer protection (WSS, HTTPS), parameterized database interfaces, role-based authorization scopes, and encrypted, access-logged storage of sensitive athlete health data.

2. Select a Suitable Software Architecture

We select a Hybrid Software Architecture style to satisfy the divergent constraints of high-frequency sensor stream ingestion, real-time ML inference, relational metadata querying, and asynchronous reporting/notification tasks.

The hybrid system combines three architectural patterns:

- Microservices Architecture: Isolates domain systems (Authentication, Athlete Profiles, Data Ingestion, Risk Prediction, Alerting, Analytics) into independent, REST-compliant services. This allows cost-effective, independent scaling and ensures a fault in one service (e.g., the Analytics service crashing during a heavy report export) does not bring down live risk monitoring.
- Event-Driven Architecture (EDA): Orchestrates asynchronous, high-throughput tasks such as sensor stream ingestion, risk-score recomputation, and alert dispatch. A message broker (Kafka) decouples ingestion from inference to protect live-session throughput.
- Layered Client-Server (Streaming + REST): Dictates the live monitoring engine. A dedicated streaming layer synchronizes sensor telemetry and pushes real-time risk alerts, while REST/JSON APIs serve standard CRUD operations for profiles and history.

2.1. Technical Justification for Selected Hybrid Architecture

Architectural Need	Monolith / Basic MVC Limitations	Hybrid (Microservices + EDA) Advantage
High-frequency sensor ingestion	Traditional MVC server threads block on HTTP requests, causing thread overhead and frequent database polling bottlenecks during live sessions.	Kafka streams absorb bursty sensor telemetry, decoupling ingestion from database writes and minimizing direct database hits.
Real-time ML risk inference	Running inference synchronously inside a monolithic request cycle slows down all concurrent users and risks timeouts under load.	A dedicated Risk Prediction Service consumes feature events asynchronously, scales independently, and publishes results without blocking ingestion.
Heavy background analytics & reports	Generating season-long trend reports blocks CPU loops on the server, slowing down live monitoring for other coaches.	Reports are queued via RabbitMQ/Kafka; workers process rendering asynchronously and notify the coach once complete.
High durability under scaling pressure	A memory leak in one module can compromise auth, dashboards, and alerts	Each microservice scales and fails independently; an Analytics Service

Architectural Need	Monolith / Basic MVC Limitations	Hybrid (Microservices + EDA) Advantage
	simultaneously, causing global service interruption.	outage does not interfere with active risk alerting.

3. Draw the Software Architecture Diagram

The diagram below represents the software architecture. It illustrates client-side data capture, API routing, microservice decomposition, streaming pipelines, and target databases that collaborate to manage the athlete injury-risk monitoring workflow.

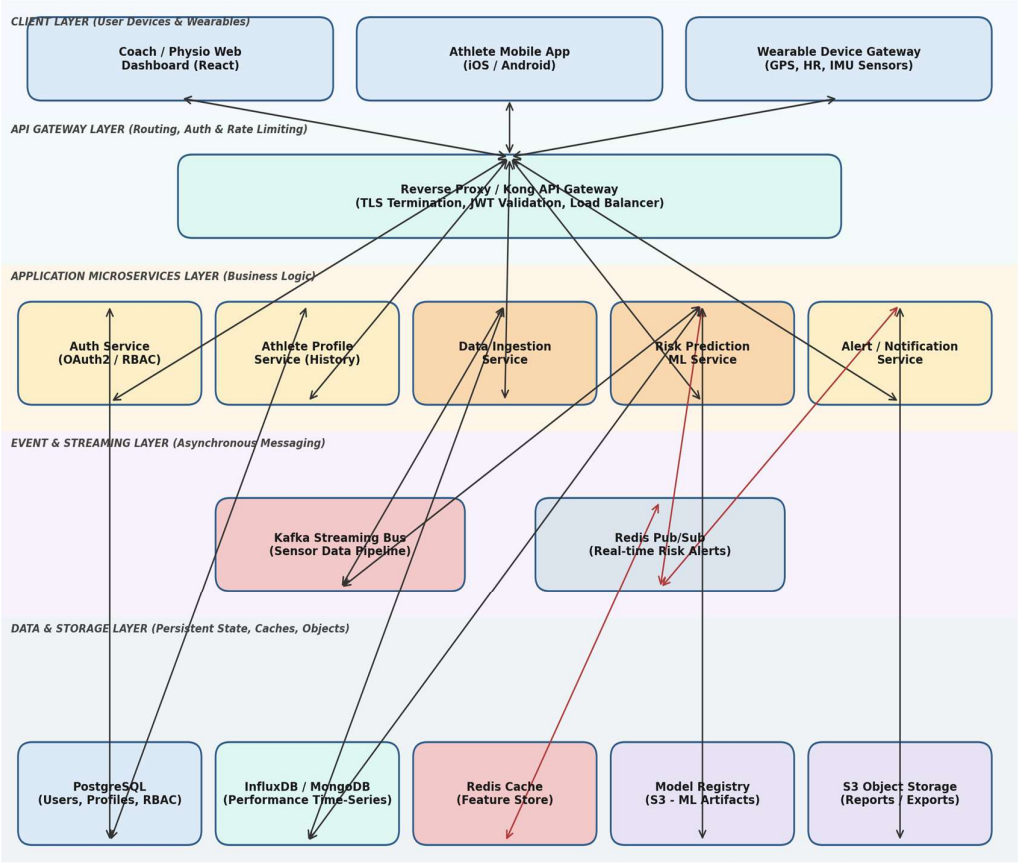

Figure 3.1: Complete Software Architecture Blueprint for the Injury Risk Prediction Platform

The architecture decomposes the platform into distinct, independent tiers. Below, we describe the integration paths:

- Wearable devices and mobile apps stream telemetry through a Device Gateway to the Data Ingestion Service via the API Gateway; standard metadata operations (profiles, history, settings) use REST HTTP APIs.
- The API Gateway manages authentication via JWT tokens, handles SSL/TLS termination, and conducts rate-limiting checks to prevent Denial of Service (DoS) attacks.
- The Kafka streaming bus carries normalized sensor events from ingestion to the Risk Prediction ML Service, decoupling high-volume telemetry from downstream processing.
- Redis Pub/Sub acts as a lightweight event broker, broadcasting computed risk alerts to connected coach dashboards in real time and caching the online feature store.

- Persistent storage is split into PostgreSQL (ideal for ACID transactions, user management, and authorization controls), InfluxDB/MongoDB (ideal for high-volume performance time-series data), and AWS S3 for ML model artifacts and exported reports.

4. Explain Components and Their Interactions

A comprehensive description of individual system components and their communications governs the architectural soundness of this platform.

4.1. Component Roles and Responsibilities

Component Name	Core Responsibilities	Technologies
React Coach Dashboard & Athlete App	Displays live risk scores, training load trends, and alert banners; captures manual RPE/wellness entries.	React SPA, React Native, WebSockets
Kong API Gateway	Validates JWT tokens, checks user session state, runs rate-limit filters, and forwards connections to internal microservice channels.	Kong, Lua, Nginx
Auth Service	Manages user registration, credential security, logins, multi-factor tokens, and issues cryptographic JWT session keys.	Node.js, Express, bcrypt, JWT
Data Ingestion Service	Normalizes heterogeneous wearable payloads, buffers offline data, and publishes structured events onto the streaming bus.	Python/Node.js, Kafka Producer SDK
Risk Prediction ML Service	Consumes feature events, runs trained injury-risk models on rolling windows, and emits risk scores with confidence bands.	Python, Scikit-learn/PyTorch, Kafka Consumer
Alert / Notification Service	Evaluates risk thresholds, dispatches WebSocket pushes and mobile notifications, and logs coach acknowledgements.	Node.js (WS), Redis Pub/Sub, FCM/APNs

4.2. Component Interactions and Data Flow

Communication protocols are optimized for each channel's requirements:

- HTTP/REST with JSON: Used for CRUD operations such as logging in, updating athlete profiles, managing injury history, and retrieving trend reports. This ensures standard, simple client-server messaging for non-real-time interactions.
- Streaming (Kafka): Used for high-frequency sensor telemetry. Devices publish readings through the Data Ingestion Service, which the Risk Prediction Service consumes as normalized feature events.
- WebSocket (WSS): Used for pushing live risk alerts to coach dashboards the moment a threshold is crossed, without requiring polling.

- AMQP/Kafka (Background Jobs): Handles asynchronous work such as season-trend report generation. The system queues the job, renders it in the background, writes to S3, and notifies the requester via WebSocket.
- gRPC / Internal REST: Used for internal microservice communication (e.g., the Risk Prediction Service validating athlete access via the Auth Service), using short-lived TCP paths to minimize latency.

4.3. System Workflows & Sequence Flow

The sequence below illustrates the real-time monitoring workflow, tracking a performance reading from a wearable sensor through ML inference and coach alerting:

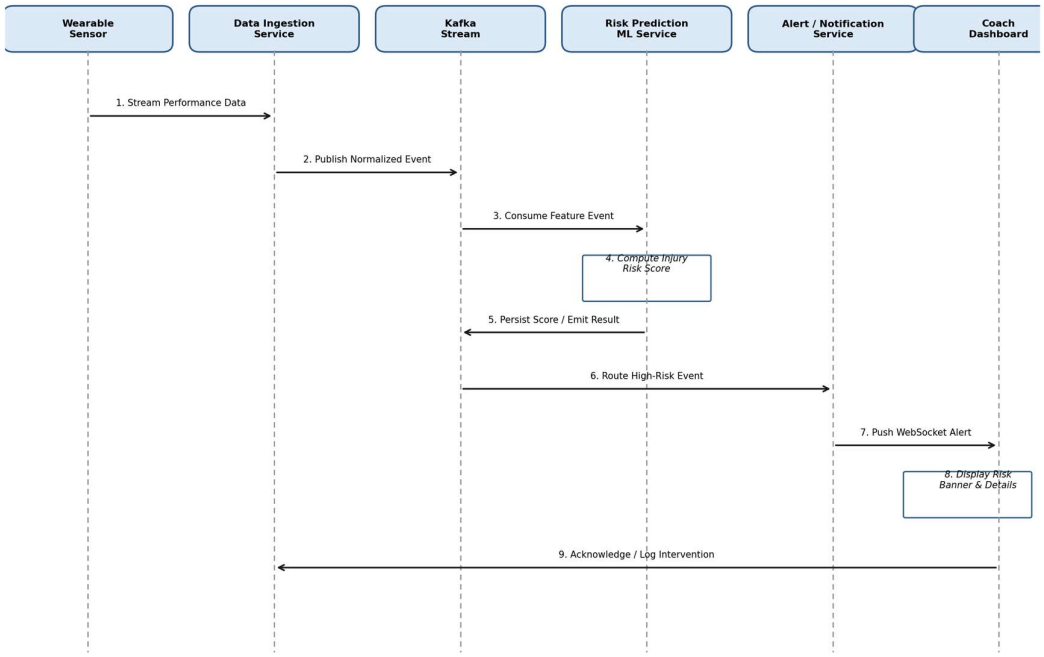

Figure 4.1: Real-time Data Ingestion and Injury Risk Alerting Workflow

5. Discuss Quality Attributes

Evaluating how our architecture addresses critical quality metrics helps verify system viability:

Quality Attribute	Architectural Strategy	Concrete Implementation Details
Scalability	Horizontal scaling of ingestion/inference nodes, sharding databases, caching feature data, and isolating heavy services.	Ingestion and ML nodes scale via Docker containers behind an Application Load Balancer. Kafka topics are partitioned by team/athlete cohort, and the time-series database is sharded by athlete UUID to prevent indexing bottlenecks.
Security	Securing connections, sandboxing processes, sanitizing data inputs, and maintaining access logs for sensitive health data.	All external connections run TLS 1.3. Athlete health fields are field-level encrypted at rest. User tokens are validated via JWT at the Gateway, and all database operations use prepared statements to prevent SQL injection.
Performance	Using edge caching, local dashboard caching, and running heavy analytics in the background.	Static dashboard assets load via CDN; the React GUI caches recent risk scores locally for instant rendering. Long-running trend reports are offloaded to background workers via the message queue.
Reliability	Using replicated streaming topics, database replication, and write-ahead logging for score persistence.	Kafka replicates sensor events across brokers to prevent loss. PostgreSQL and the time-series database replicate in real time with automated failover. The Risk Prediction Service caches un-persisted scores in Redis to prevent loss during transient failures.
Maintainability	Enforcing service isolation, clean API standards, and automated test pipelines.	Services are separated by domain boundary (Auth, Ingestion, Prediction, Alerting). Contracts are defined using OpenAPI specifications, and CI/CD pipelines run unit and integration tests before deployment.
Availability	Multi-region failovers, health checks, auto-healing systems, and database clustering.	Containers run on Kubernetes with configured health probes; unresponsive nodes are auto-terminated and replaced. Database replica sets automatically promote secondary nodes if a primary fails, keeping monitoring online.

6. Justify Architectural Decisions

Every architectural structure represents a balance of constraints. Below, we justify our choices, discuss the trade-offs we considered, and outline our plans for future scalability.

6.1. Rationale Behind Design Choices

Our core architectural choice is the division between real-time streaming risk inference and relational metadata storage. Using a Kafka-based streaming pipeline ensures that sensor ingestion and ML inference occur independently of our relational database structures, preventing database lock congestion during high-volume training sessions. By offloading heavy tasks like report generation and historical trend analysis to asynchronous workers, we maintain a highly responsive coach-facing dashboard experience.

6.2. Trade-off Analysis Matrix

Decision	Selected Path	Advantages Gained	Drawbacks / Mitigations
Inference Strategy	Streaming, near-real-time ML inference (Kafka + online feature store)	Sub-2-second risk visibility, enabling in-session coaching decisions and immediate intervention.	Higher infrastructure complexity than batch scoring. Mitigated by a managed online feature store and model-serving autoscaling.
Database Selection	Polyglot Storage (PostgreSQL + InfluxDB/MongoDB)	PostgreSQL handles structured metadata (users, RBAC, injury history) with ACID guarantees, while InfluxDB/MongoDB efficiently stores high-volume sensor time-series data.	Higher operational cost and complexity. Mitigated by containerising both databases and orchestrating them via Kubernetes.
Real-time Alert Broadcast	Redis Pub/Sub Broker Sync	Provides fast, low-overhead alert delivery across all active dashboard connections, supporting many concurrently monitored teams.	Redis is an in-memory store, so sudden restarts can clear active subscriptions. Mitigated by persisting alert history in PostgreSQL.

6.3. Future System Integration & Long-term Maintainability

Our chosen architecture supports future enhancements through its modular API design. The API gateway separates external web/mobile interfaces from internal services, allowing us to implement features like AI-assisted rehabilitation planning or automated video-based movement analysis by adding new microservices without modifying the core ingestion or prediction engine. All service contracts use OpenAPI specifications, ensuring that teams can work independently and develop components in different programming languages as the platform grows.

Appendix: Architecture Design Assignment Rubric Matrix

This section includes the assessment rubrics for evaluation reference:

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram & Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction & Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification & Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.